Vaughn Darreill B. Pereyra
IT3R12

Activity 1: Cloud Service Model Comparison

Category	**IaaS (Infrastructure as a Service)**	**PaaS (Platform as a Service)**	**SaaS (Software as a Service)**
Description	Provides virtualized computing resources over the internet such as servers, storage, and networking. The user manages the OS and applications while the provider manages the physical infrastructure.	Provides a ready-made platform for developers to build, test, and deploy applications without managing the underlying hardware or OS.	Delivers fully functional software applications over the internet. Users access the software through a browser or app without handling any infrastructure or platform concerns.
Examples	Amazon Web Services (AWS) EC2, Microsoft Azure Virtual Machines, Google Compute Engine	Google App Engine, Microsoft Azure App Service, Heroku	Google Workspace, Microsoft 365, Salesforce, Zoom
Advantages	High flexibility and control over the environment. Scalable on demand. No need to buy or maintain physical hardware.	Speeds up development by removing infrastructure setup. Developers can focus only on writing code. Built-in tools for testing and deployment.	No installation or maintenance needed by the user. Accessible from anywhere with internet. Automatic updates handled by the provider.

Limitations	Requires technical knowledge to manage servers and configurations. Security depends on how well the user configures the environment.	Less control over the underlying infrastructure. Can be limiting if the application needs a specific OS or environment setup.	Limited customization. Data is stored on the provider's servers. Dependent on internet connection and provider uptime.
Best Scenario	A startup that wants to host its own web server without buying physical hardware. They need full control over the OS and software stack.	A development team building a web application who wants to focus on coding and not worry about server setup or maintenance.	A company that needs employees to collaborate using email, documents, and video calls without setting up any software internally.

Justification

IaaS is most suitable when an organization needs full control over its computing environment. For example, a company running a custom server setup or hosting a database with specific OS requirements would benefit from IaaS because they can configure everything from scratch without owning physical machines.

PaaS is most suitable for development teams that want to build and deploy applications quickly. Since the platform handles the infrastructure and runtime environment, developers can focus on writing and testing their code without worrying about OS updates or hardware management.

SaaS is most suitable for end users or businesses that just need a tool to get work done. A school using Google Workspace for emails and documents does not need to manage any servers or software. They just log in and start using it.

Activity 2: Microservices Architecture Design

Chosen Application: Food Delivery App

Identified Microservices

1. User Service - handles user registration, login, and profile management.
2. Restaurant Service - manages restaurant listings, menus, and availability.
3. Order Service - handles order creation, tracking, and order history.
4. Payment Service - processes payments and manages transaction records.
5. Delivery Service - assigns delivery riders and tracks delivery status in real time.

Architecture Overview

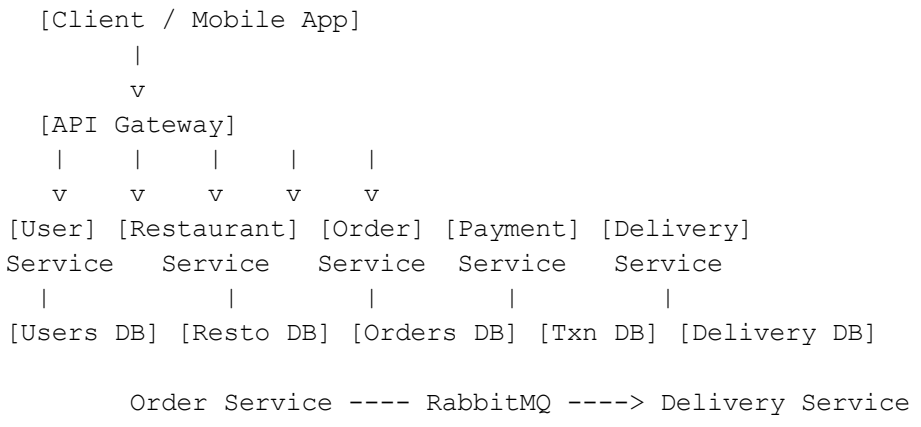
The client (mobile app or browser) sends all requests to the API Gateway. The API Gateway acts as the single entry point and routes each request to the correct microservice.

Each microservice operates independently and has its own database. They communicate with each other using REST for synchronous requests and RabbitMQ for asynchronous tasks like sending notifications after an order is placed.

Component	**Details**
Client	Mobile app or web browser used by customers and restaurants
API Gateway	Single entry point that routes requests to the correct service. Also handles authentication and rate limiting.
User Service	Manages accounts and authentication. Has its own users database.

Restaurant Service	Manages restaurant data and menus. Has its own restaurants database.
Order Service	Creates and tracks orders. Has its own orders database. Communicates with Payment and Delivery services.
Payment Service	Handles payment processing. Has its own transactions database.
Delivery Service	Assigns riders and tracks delivery. Has its own delivery database.
Message Queue	RabbitMQ used for asynchronous communication between Order Service and Delivery Service
Communication	REST (HTTP) for synchronous calls between services, RabbitMQ for async tasks like notifications

Simple Architecture Diagram (Text Form)



Handling Failure in One Service

If one service goes down, for example the Delivery Service fails, the rest of the application continues to work normally. Users can still browse restaurants, place orders, and complete payments through the other services. The Order Service would detect that the Delivery Service is unavailable and use a circuit breaker pattern to stop sending requests to it temporarily instead of waiting forever.

The system can also use a message queue like RabbitMQ so that when the Delivery Service comes back online, it picks up the pending delivery assignments from the queue and processes them. This way no data is lost even when the service was down.

Centralized monitoring tools like Prometheus and Grafana would alert the team when a service goes down so they can respond quickly. Because each service is deployed in its own Docker container, the team can restart or redeploy only the Delivery Service without touching any other part of the application.